

Inteligencia artificial avanzada para la ciencia de datos (Gpo 601)



Momento de Retroalimentación: Redefinición de los datos

(Gpo 601)

Edgar Osvaldo Navarro García A01644488

Septiembre 4, 2026

Introducción

El dataset de REHAB recolecta datos sobre pacientes sobrevivientes de un Accidente Cerebrovascular. Consiste en 32 archivos los cuales representan los datos recolectados por 2 distintos conjuntos de sensores al momento de realizar 16 movimientos de rehabilitación. Estos datos son mayormente series temporales que demuestran el estado de distintos canales de los sensores al momento de realizar el movimiento. Cada movimiento cuenta con un total de 11 canales y estos recolectaron 880 puntos a una frecuencia de 50 hz correspondiendo a 17.6 segundos en caso de que el movimiento se hubiera terminado antes de tiempo se llenaron los puntos faltantes con 0. A partir de este dataset se busca predecir qué tipo de movimiento se está haciendo referencia utilizando estas series de tiempo. A continuación se presenta el proceso realizado para tratar los datos partiendo desde los datos raw y la implementación del algoritmo del centroide más cercano utilizando ventanas temporales para reemplazar las series de tiempo.

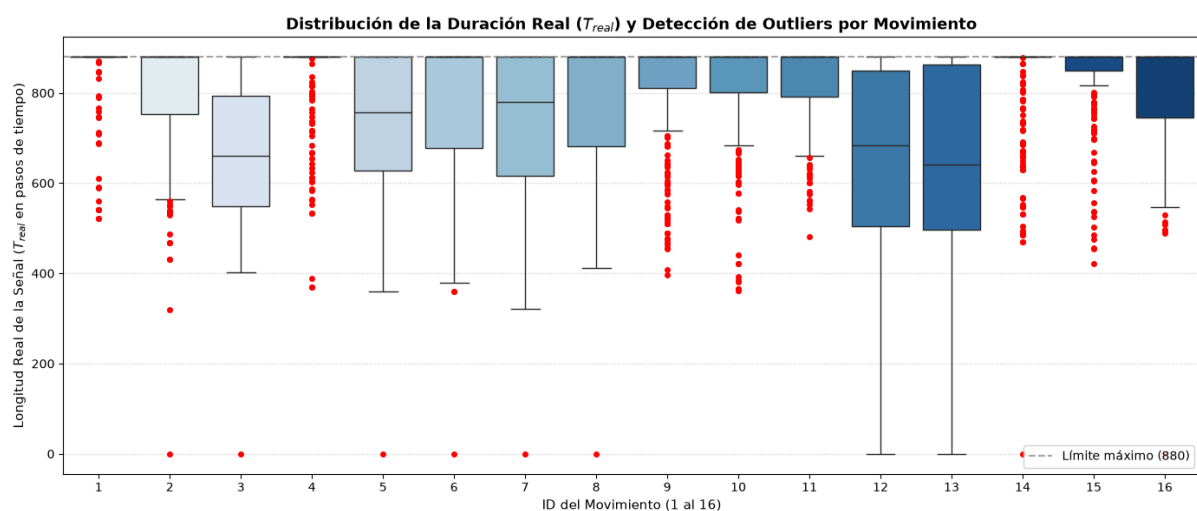
Tratamiento de datos

Búsqueda y reemplazo de duplicados, datos nulos y muestras corruptas

Lo primero que se realizó fue probar que los tamaños de las muestras a utilizar tuvieran las mismas dimensiones y que se tuvieran el mismo tamaño de muestra para los distintos sensores. Afortunadamente se tenían las mismas cantidades de muestras y no se requirió realizar un tratamiento para esta parte. Después se revisaron todos los datasets de los archivos en búsqueda de nulls o infinitos. Esto nos dio como resultado que no existe ningún dato nulo o infinito. Utilizando numpy se busco en cada archivo el número de registros duplicados para los cuales primero se concatenaron los datos de ambos grupos de sensores y después se realizó la búsqueda teniendo un total de 716 registros duplicados entre todos los

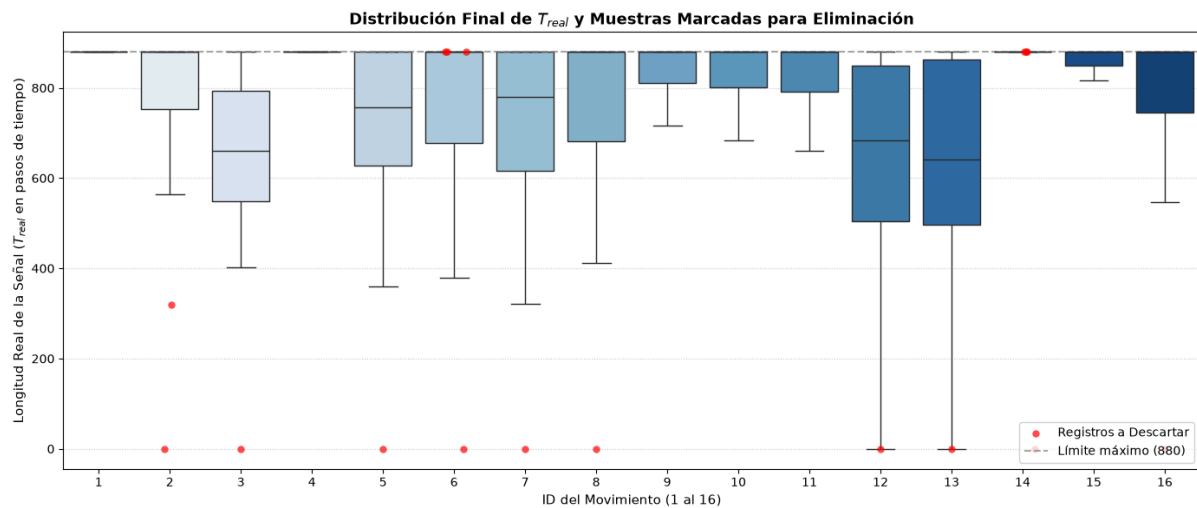
movimientos representando el 7.76% de los registros guardados. En este caso al estos ser registros duplicados se decidió eliminarlos del dataframe.

Pero también es necesario checar la proporción de ceros dentro de nuestros archivos al haber sido utilizados para agregar un padding y existe la probabilidad de encontrar canales muertos con ellos. Para esto se contaron el número de ceros por canal y se contaron un total de 6153769 representando al 89.65% de los datos registrados en las series de tiempo. En este caso el número es tan grande debido a que pudieron haber registros muertos o con algún problema en algún sensor, y también por el padding agregado para hacerlos todos series de tiempo de 880 puntos. Para arreglar este problema se revisaron las varianzas de todos los registros y se encontraron 16 registros que no tenían casi ninguna varianza lo cual fue clasificado como un registro con canal muerto y posteriormente eliminado. A su vez se graficaron las longitudes reales de los movimientos en un boxplot para revisar si había datos fuera de lo común que pudiera afectar de gran manera al modelo.



A Través de esta gráfica se observó, que existían varios movimientos con muchos valores atípicos utilizando un IQR normal y en este caso eliminarlos podría significar un gran desbalance en nuestro dataset. Por lo tanto, se decidió utilizar un IQR modificado para detectar únicamente casos muy extremos en los cuales están por debajo del 40% de la

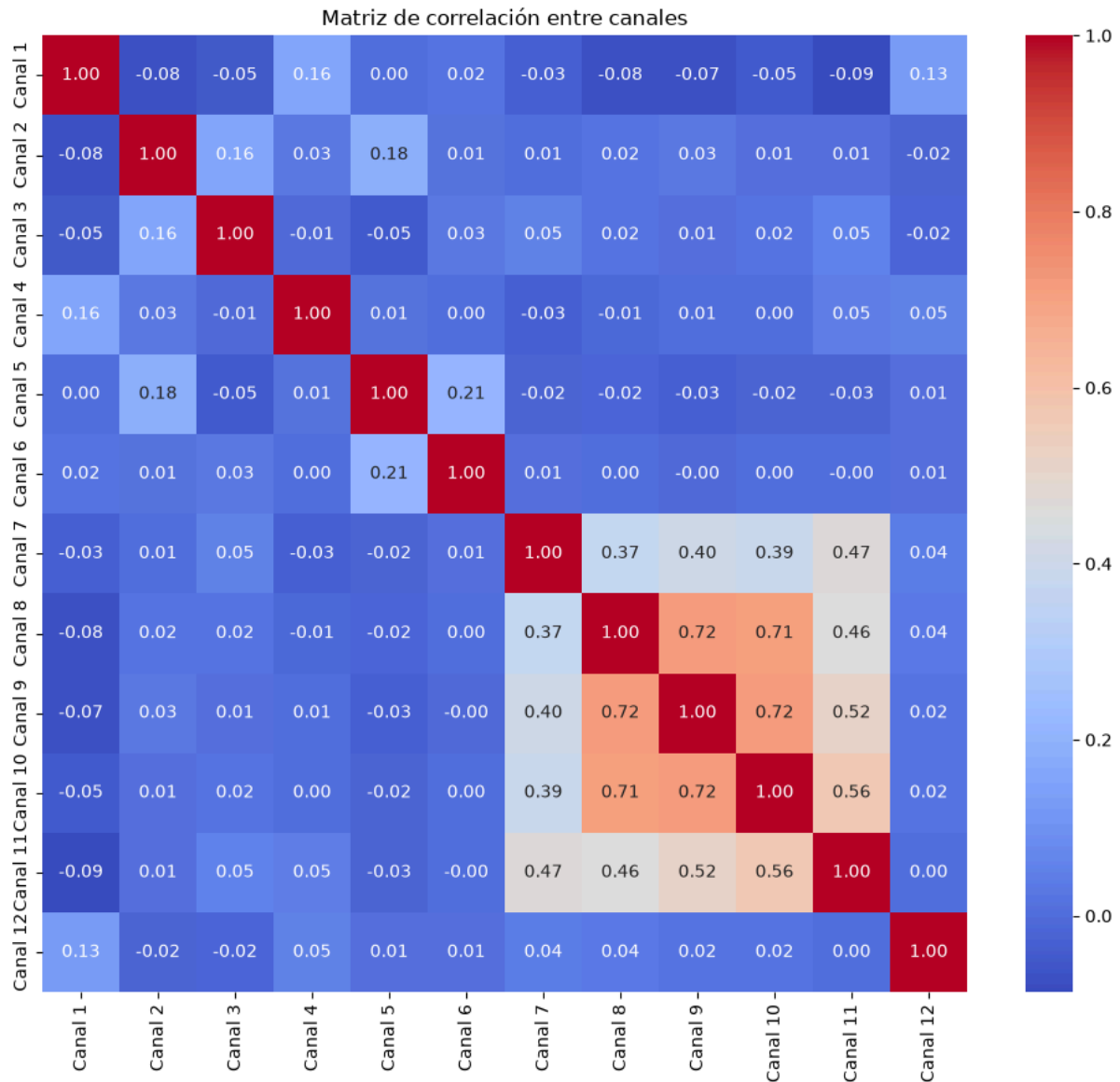
mediana. En este caso se detectó únicamente un valor el cual cumplía con esta condición, después se graficaron estos puntos en un boxplot junto con los registros a eliminar por canal muerto.



Transformación de datos

Para empezar la transformación, lo primero que se realizó fue checar el tamaño de la muestra en los distintos movimientos. Afortunadamente se pudo observar que las variaciones no fueron muy altas siendo el movimiento con más muestras el movimiento 8 con 383 y el de menos muestras el movimiento 3 con 171. Tras realizar esto se decidió unir todos los sensores en un solo data frame y separar sus etiquetas obteniendo un dataset de (3883, 880, 12) para los datos y otro de (3883) con las etiquetas.

Después se revisaron los estadísticos básicos considerando el padding de ceros. Para considerarlo tan solo se tomaron en cuenta los valores de la muestra hasta que se detecta que tienen ese padding considerando únicamente la porción del movimiento captado. Viendo las estadísticas se decidió normalizar los datos para poder tener una varianza de 1 y poder entrenar el modelo de mejor manera. También se analizó la correlación entre las variables obteniendo el siguiente mapa de calor:



Atravez de este mapa de calor se puede observar que los canales del segundo grupo de sensores estaba muy correlacionado lo cual puede significar que estos canales tienen medidas muy similares. Por esta razón se decidió unir los canales 8, 9 y 10 en uno solo canal promediado obteniendo un dataset de (3883, 880, 10).

Algoritmo de clasificación implementado manualmente

En este caso se implementó el algoritmo del centroide más cercano al este tener una baja dificultad de implementación y al este poder clasificar muestras dependiendo de sus datos de entrenamiento. En esencia el algoritmo consiste en usar todas las muestras de un clasificador y crear un nuevo punto con las medias de los canales de la muestra. Este punto generado sería el centroide y esto se realiza para cada clasificador. Una vez se tienen los centroides estos son usados para predecir nuevas muestras las cuales lo único que necesitan hacer es medir la distancia entre todos los centroides y clasificar la muestra con el centroide respecto al cual existe la menor distancia.

Separación de línea de Tiempo

Nuestro dataset de entrenamiento tenía la forma (3106, 880, 10) donde el 880 era una línea de tiempo. En este caso se decidió utilizar ventanas en lugar de esta línea de tiempo. Cada ventana tiene un tamaño de 80 puntos y en esta se recolectan los datos promedio de la ventana de la media, desviación estándar, mínimo, máximo y error de cada canal. Obteniendo un nuevo dataset de entrenamiento con la siguiente forma (3106, 550). Y lo mismo se aplicó para el dataset de prueba obteniendo un forma final de (777, 550).

Calculo de centroides

Para nuestro dataset se quieren predecir un total de 16 movimientos distintos, por lo tanto se crearon 16 centroides uno para cada movimiento usando el dataset de entrenamiento. Para crear un centroide primero se obtienen todas las muestras en el dataset sobre un movimiento en específico utilizando

Python

```
for c_idx, clase_val in enumerate(clases_unicas):  
  
    indices_clase_c = np.where(y_train == clase_val)[0]  
  
    muestras_clase_c = X_train_feat[indices_clase_c]
```

Después, se suman todos los valores de las muestras del movimiento, se guardan y se divide entre el número de muestras totales del movimiento obteniendo un centroide con 550 columnas.

Python

```
# Sumatoria manual por característica  
  
for i in range(conteo_muestras_clase):  
  
    for j in range(num_features):  
  
        suma_caracteristicas[j] += muestras_clase_c[i, j]  
  
  
# Promedio por característica  
  
centroide_c = suma_caracteristicas / conteo_muestras_clase  
  
centroides[c_idx] = centroide_c
```

Esto se repite para cada movimiento obteniendo al final 16 centroides cada uno con 550 columnas.

Predicción de clasificadores

Para las predicciones, lo que se realiza es para cada muestra a testear, se obtiene la distancia euclidiana entre la muestra con sus medidas contra las medidas de todos los centroides. Obteniendo en total para cada muestra a testear 16 distancias distintas, después utilizando argmin obtenemos la distancia vectorial mínima de entre todas las distancias hacia

los centroides. Y la distancia mínima será la tag que se le otorga a la muestra de test. Al utilizar este método se obtuvo una precisión de 58.04%.